SmartCampus

Guia de ejecucion y evidencias del taller

Proyecto Big Data I · Taller evaluable 01

Asignatura	**Proyecto Big Data I — Grado en Ciencia de Datos**
Universidad	Universidad Europea de Valencia
Integrantes	Luis Wesinger · Yiao Tang
Repositorio	github.com/luiswesinger/practica01
Entorno probado	Docker 29.1.2 · Docker Compose v2
Fecha	14 de octubre de 2026

Este documento recoge, en un solo lugar, los pasos necesarios para levantar el proyecto desde cero y las capturas que demuestran su funcionamiento. Las capturas proceden de una ejecucion real sobre el entorno descrito.

1. Resumen del estado

El taller se compone de cuatro retos. Todos estan implementados y verificados sobre el entorno Docker real.

Reto	Contenido	Estado
A	Infraestructura con Docker Compose (tres servicios)	Completado
B	Carga, almacenamiento y analisis con pandas y SQLAlchemy	Completado
C	API Flask: GET /health, GET /lecturas, POST /lecturas	Completado
D	Prueba de integracion de extremo a extremo	Completado

1.1 Los tres servicios

Servicio	Imagen base	Contenedor	Puerto en el host
db	postgres:16-alpine	smartcampus_db	no se publica
api	python:3.13-slim + Flask	smartcampus_api	5000
jupyter	quay.io/jupyter/minimal-notebook	smartcampus_jupyter	8888

Nota: el puerto de PostgreSQL no se publica al equipo anfitrion de forma deliberada. Solo lo consumen los servicios api y jupyter a traves de la red interna de Compose, de modo que no puede entrar en conflicto con otros PostgreSQL que ya esten instalados en la maquina.

2. Como ejecutar el proyecto

2.1 Requisitos previos

- Docker Desktop (o Docker Engine con el plugin docker compose), en ejecucion.
- Conexion a Internet para el primer arranque: hay que descargar las imagenes base.
- Los puertos 8888 y 5000 libres en el equipo anfitrion.

No hace falta instalar Python, PostgreSQL ni Jupyter en el equipo: todo se ejecuta dentro de los contenedores.

2.2 Arrancar el entorno

```
$ git clone https://github.com/luiswesinger/practica01.git
$ cd practica01
$ docker compose up --build
```

El primer arranque tarda entre 10 y 20 minutos porque descarga las imagenes base e instala las dependencias. Los siguientes son mucho mas rapidos, ya que Docker reutiliza las capas en cache.

2.3 Comprobar que los tres servicios estan activos

```
$ docker compose ps
```

Deben aparecer tres contenedores en ejecucion. Los servicios db y jupyter figuran ademas como (healthy).

2.4 Si un puerto esta ocupado

En macOS el puerto 5000 suele estar ocupado por el sistema (el Receptor de AirPlay). Basta con cambiar el puerto publicado:

```
$ cp .env.example .env
# editar .env y ajustar:
JUPYTER_PORT=8889
API_PORT=5001
```

Se modifica unicamente el lado izquierdo del mapeo. El puerto de la derecha es el que usa el proceso dentro del contenedor y no debe cambiarse.

2.5 Acceder a los servicios

Servicio	Direccion	Para que
Jupyter	http://localhost:8888	Ejecutar el notebook y ver los analisis
API	http://localhost:5000/health	Comprobar la API
API	http://localhost:5000/lecturas	Consultar las lecturas

2.6 Ejecutar el notebook y hacer la prueba de integracion

En Jupyter, abrir notebooks/analyse_data.ipynb y ejecutar todas las celdas en orden (menu Kernel → Restart Kernel and Run All Cells). El notebook tarda unos segundos y es repetible: puede ejecutarse varias veces sin acumular datos duplicados.

3. Evidencias de funcionamiento

Las cuatro evidencias obligatorias del enunciado, con las capturas obtenidas durante la ejecucion.

3.1 Evidencia 01 · Servicios en ejecucion

Que debe verse: los tres servicios del taller levantados y comunicados.

```
$ docker compose ps
```



Fig. 1 — Los tres servicios en ejecucion · evidencias/01_servicios_compose.png

- **Servicios activos:** Los tres contenedores estan en estado Up.
- **db:** 5432/tcp, sin publicar: PostgreSQL solo es accesible desde la red interna.
- **db** y jupyter figuran como (healthy): sus comprobaciones de salud pasan.
- **Mapeos** 5001→5000/tcp y 8889→8888/tcp: puertos publicados en el equipo anfitrion (se usaron puertos alternativos porque los de por defecto estaban ocupados).

3.2 Evidencia 02 · Bases de datos

Que debe verse: las consultas y los recuentos de registros en PostgreSQL y en SQLite.

El notebook lee sensores.csv y carga las 192 lecturas primero en PostgreSQL y despues en SQLite, reutilizando exactamente la misma funcion. A continuacion obtiene los dos analisis obligatorios.

Analisis obtenidos en PostgreSQL

La temperatura media se agrega en SQL (GROUP BY + AVG); el CO2 medio, en pandas. La agregacion en el motor evita traer filas innecesarias al DataFrame.

```
print("\nAverage CO2 per building (In-memory pandas):")
print(df_co2_pg)
print(f"\nBuilding with highest mean CO2: Building {top_co2_pg_edificio} ({top_co2_pg_valor} ppm)")
```

```
Average temperature per building (Engine-side SQL):
  edificio  temp_media
0        A       22.63
1        B       23.68
2        C       24.44
3        D       23.08

Average CO2 per building (In-memory pandas):
  edificio  co2_medio
0        A      822.4
1        B     1002.9
2        C     1202.4
3        D     1349.3

Building with highest mean CO2: Building D (1349.3 ppm)
```

Fig. 2 — Analisis de temperatura y CO2 · evidencias/02a_postgresql_analisis.png

Recuento de registros en SQLite

El mismo DataFrame se carga en SQLite cambiando unicamente el engine. El recuento se verifica con una asercion, de modo que el notebook falla si el numero de filas no es el esperado.

```
Building with highest mean CO2: Building D (1349.3 ppm)
```

SQL Lite

```
[6]: sqlite_engine = create_engine(SQLITE_URI)

# Clean up table in SQLite
with sqlite_engine.begin() as conn:
    conn.execute(text("DROP TABLE IF EXISTS lecturas;"))

# Append data
df.to_sql("lecturas", con=sqlite_engine, if_exists="append", index=False)

# Verify row count
with sqlite_engine.connect() as conn:
    sqlite_count = conn.execute(text("SELECT COUNT(*) FROM lecturas")).scalar()

assert sqlite_count == 192, f"Incorrect row count in SQLite: {sqlite_count} != 192"
print(f"Records successfully loaded into SQLite: {sqlite_count}")
```

```
Records successfully loaded into SQLite: 192
```

Fig. 3 — Carga en SQLite y verificacion del recuento · evidencias/02b_sqlite_recuento.png

Resultados identicos en los dos motores

La funcion cargar_y_resumir(engine, data) se ejecuta contra los dos engines. Los resultados coinciden fila por fila, lo que demuestra que la logica no depende del fabricante de la base de datos.

```
print("\nComparative CO2 Results:")
print("--- PostgreSQL ---")
print(res_pg["co2_medio"])
print("--- SQLite ---")
print(res_sqlite["co2_medio"])
```

```
Comparative Temperature Results:
--- PostgreSQL ---
  edificio  temp_media
0        A       22.63
1        B       23.68
2        C       24.44
3        D       23.08
--- SQLite ---
  edificio  temp_media
0        A       22.63
1        B       23.68
2        C       24.44
3        D       23.08

Comparative CO2 Results:
--- PostgreSQL ---
  edificio  co2_medio
0        A      822.4
1        B     1002.9
2        C     1202.4
3        D     1349.3
--- SQLite ---
  edificio  co2_medio
0        A      822.4
1        B     1002.9
2        C     1202.4
3        D     1349.3
```

Fig. 4 — Comparativa de resultados entre motores · evidencias/02c_comparativa_motores.png

Resultados obtenidos

Edificio	Temperatura media	CO2 medio
A	22,63 °C	822,4 ppm
B	23,68 °C	1002,9 ppm
C	24,44 °C ← la mas alta	1202,4 ppm
D	23,08 °C	1349,3 ppm ← el mas alto

Nota sobre el edificio A: su media exacta es 22,625, justo en la mitad de una centesima. PostgreSQL redondea a 22,63 y pandas a 22,62, porque aplican reglas de redondeo distintas. La diferencia de 0,01 no es un error de calculo.

3.3 Evidencia 03 · API

Que debe verse: la respuesta JSON de GET /lecturas y la ejecucion satisfactoria de POST /lecturas.

GET /lecturas

Devuelve las lecturas reales almacenadas en PostgreSQL. El enunciado prohibe expresamente devolver una lista escrita a mano en Python.

```
$ curl -i http://localhost:5000/lecturas
```



Fig. 5 — GET /lecturas devuelve HTTP 200 y el JSON · evidencias/03a_api_get_lecturas.png

POST /lecturas con un cuerpo valido

Se envia el ejemplo incluido en el repositorio. La API responde 201 y devuelve la lectura creada con su identificador.

```
$ curl -i -X POST http://localhost:5000/lecturas \
    -H "Content-Type: application/json" \
    --data @04_Ejemplos_pruebas/01_lectura_valida.json
```

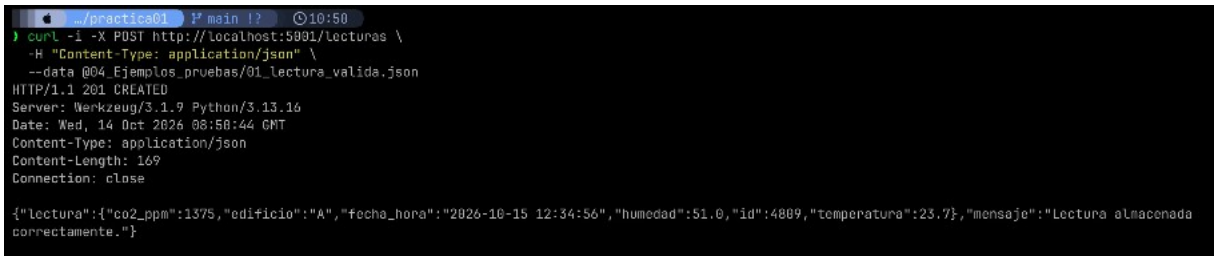


Fig. 6 — POST /lecturas correcto: HTTP 201 CREATED · evidencias/03b_api_post_201.png

POST /lecturas sin un campo obligatorio

Se envia el ejemplo al que le falta co2_ppm. La API responde 400 y detalla que campos faltan. El registro no se guarda.

```
$ curl -i -X POST http://localhost:5000/lecturas \
    -H "Content-Type: application/json" \
    --data @04_Ejemplos_pruebas/02_lectura_incompleta_HTTP400.json
```

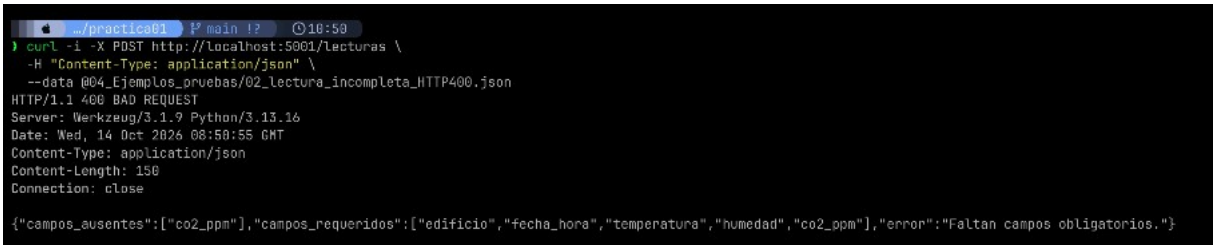


Fig. 7 — Validacion: HTTP 400 BAD REQUEST · evidencias/03c_api_post_400.png

GET /health

Comprueba el servicio y ademas el acceso real a la base de datos: ejecuta una consulta contra PostgreSQL. Si la base no responde, devuelve 503 en lugar de un 200 enganoso.

```
$ curl -i http://localhost:5000/health
```



Fig. 8 — GET /health: HTTP 200 con el estado de la base de datos · evidencias/03d_api_health.png

Consulta desde el navegador

La misma ruta GET /lecturas abierta en un navegador, para mostrar que la API es consumible directamente sin herramientas adicionales.

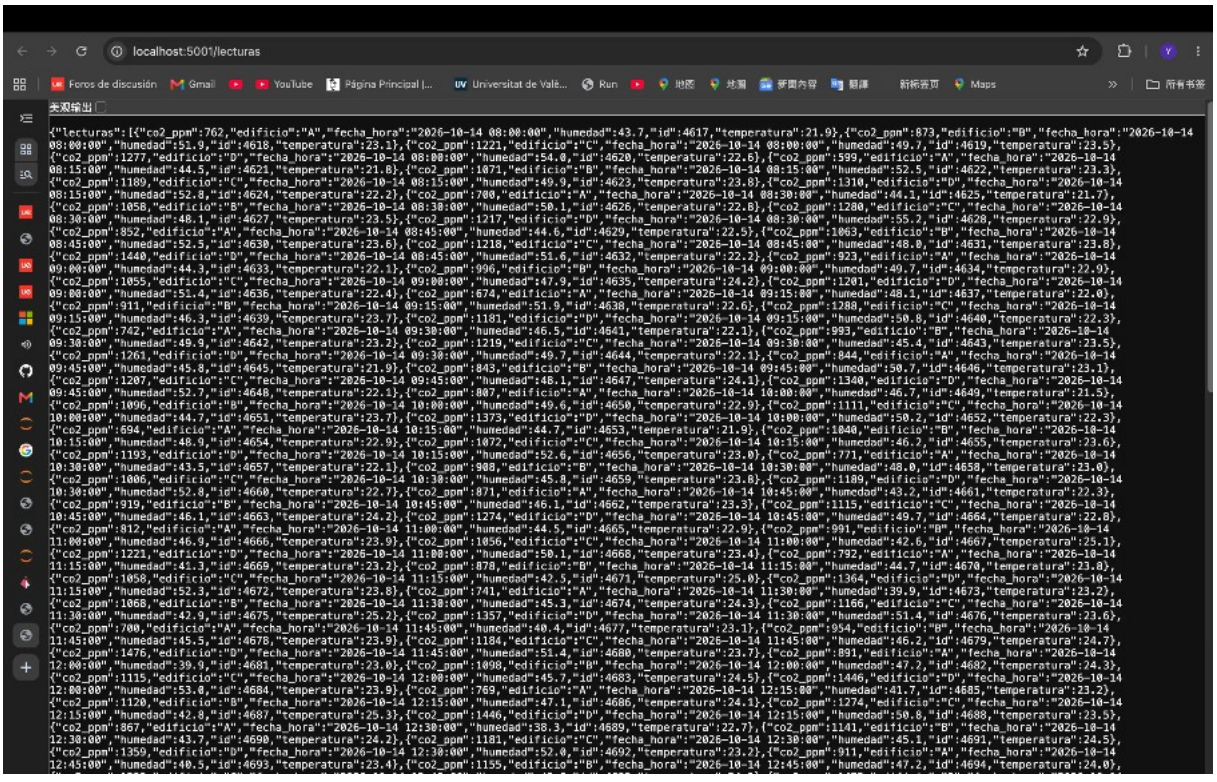


Fig. 9 — GET /lecturas visto desde el navegador · evidencias/03e_api_navegador_lecturas.png

3.4 Evidencia 04 · Integracion de extremo a extremo

Que debe verse: en Jupyter, la peticion POST realizada y la comprobacion de esa misma lectura en PostgreSQL.

Es el reto con mayor peso de la rubrica. Se ejecuta desde el notebook, con la biblioteca requests, y demuestra que los tres servicios estan realmente conectados.

```
# 1) valor de referencia, contado con SQLAlchemy (no con la API)
with pg_engine.connect() as conn:
    filas_antes = conn.execute(text('SELECT COUNT(*) FROM lecturas')).scalar()


# 2) peticion a la API, usando el NOMBRE DEL SERVICIO
respuesta = requests.post(f'{API_URL}/lecturas', json=nueva_lectura, timeout=10)
assert respuesta.status_code == 201


# 3) comprobacion de la persistencia real: se vuelve a leer la base de datos
with pg_engine.connect() as conn:
    filas_despues = conn.execute(text('SELECT COUNT(*) FROM lecturas')).scalar()
    registro = conn.execute(text('SELECT * FROM lecturas WHERE id = :id'),
                            {'id': id_creado}).fetchone()
assert filas_despues == filas_antes + 1
```

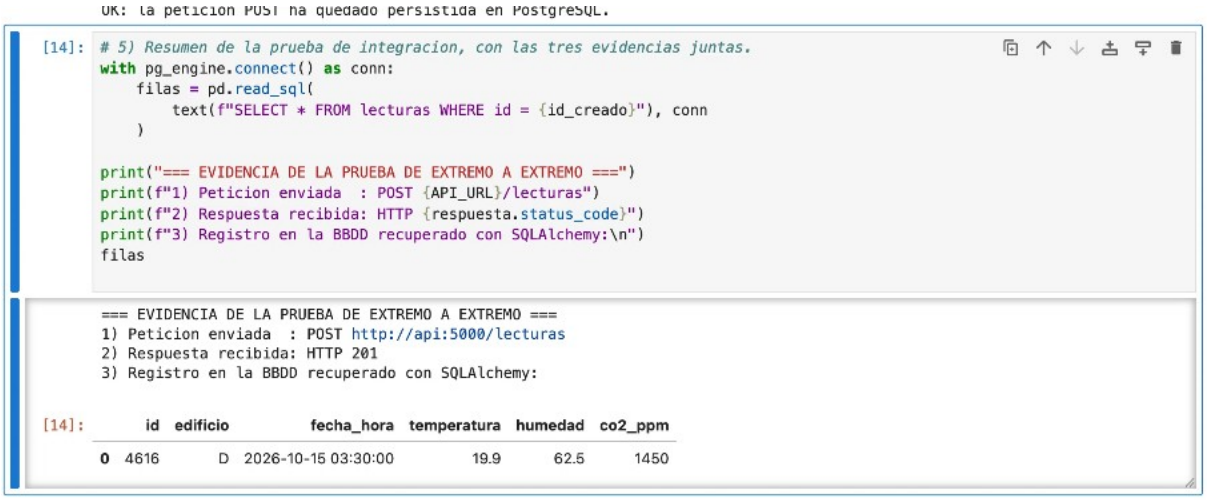


Fig. 10 — Prueba de extremo a extremo completa · evidencias/04_integracion_e2e.png

Como se interpreta

- **Paso 1:** La API se alcanza como http://api:5000, usando el nombre del servicio. Con localhost apuntaria al propio contenedor de Jupyter y fallaria.
- **Paso 2:** Se cuentan las filas con SQLAlchemy y no con la API, para que la comprobacion sea independiente del servicio que se esta evaluando.
- **Paso 3:** La API confirma la creacion devolviendo el identificador de la lectura.
- **Paso 4:** Se vuelve a consultar PostgreSQL y el numero de filas pasa de 192 a 193; los valores coinciden con los enviados.

Ensenar unicamente la respuesta JSON de la API no demostraria nada: una captura de JSON sin persistencia efectiva no prueba que la integracion funcione. Por eso la prueba vuelve a leer la base de datos. Si el registro no se hubiera guardado, el notebook fallaria con una asercion en lugar de dar un resultado enganoso.

4. Verificacion realizada

Comprobaciones ejecutadas sobre el entorno real. Los resultados son salidas reales, no ejemplos teoricos.

#	Comprobacion	Resultado
1	docker compose ps	Tres servicios activos; db y jupyter healthy
2	Esquema de la tabla lecturas	6 columnas + clave primaria + 2 indices
3	GET /health	200 · {"status":"ok","database":"ok"}
4	GET /lecturas	200 · coleccion JSON con las lecturas
5	POST /lecturas (valido)	201 · registro persistido
6	POST /lecturas (sin co2_ppm)	400 · indica los campos ausentes
7	Persistencia tras reiniciar	Los datos siguen presentes (volumen)
8	Conexion Jupyter → db por nombre	Correcta; SELECT count(*) responde
9	Notebook completo, dos veces	14 celdas sin errores; el recuento no se acumula
10	Resultados PostgreSQL vs SQLite	Identicos en los dos motores

4.1 Lista de comprobacion del enunciado

- [ ] Los tres servicios arrancan con docker compose up --build.
- [ ] El notebook se conecta a PostgreSQL y a SQLite.
- [ ] Se obtienen los resultados analiticos solicitados.
- [ ] Los tres endpoints de Flask responden como se espera.
- [ ] El POST desde Jupyter crea un registro verificable en PostgreSQL.
- [ ] Un companero podria reproducir el proyecto siguiendo solo el README.
- [ ] Se han incluido capturas y se han evitado credenciales reales.
- [ ] Se ha documentado el uso de IA (o declarado su no uso).

4.2 Problemas encontrados y solucion

Problema	Causa	Solucion
pip install falla con Permission denied	COPY conserva el modo 600 del fichero y la imagen base no ejecuta como root	COPY --chown=jovyan:users
Relation "lecturas" does not exist	El volumen no estaba vacio, asi que init.sql no se ejecuto	docker compose down -v y volver a arrancar
Puerto 5000 ocupado en macOS	Lo usa el Receptor de AirPlay del sistema	API_PORT=5001 en el fichero .env
Los recuentos no cuadran (384 en vez de 192)	Se ejecuto la carga dos veces sin vaciar la tabla	El notebook hace DELETE antes de cargar

Documento generado a partir del repositorio del proyecto. El enunciado oficial de la practica prevalece sobre cualquier indicacion recogida aqui.